Makefile (0,5 puntos)

Crea un fichero Makefile que permita generar todos los programas del enunciado a la vez y cada uno por separado. Añade una regla (clean) para borrar todos los binarios y/o ficheros objeto, y dejar solo los ficheros fuente. Los programas deben generarse si, y solo si, ha habido cambios en los ficheros fuente.

Control de errores (0,5 puntos)

Para todos los programas que se piden a continuación hay que comprobar los errores de TODAS las llamadas al sistema (excepto el write por pantalla), controlar los argumentos de entrada y definir la función Usage() si es necesario.

Ejercicio 1 (2 puntos) – Búsqueda de cadena

Haz un programa llamado **contains.c**, que indique por pantalla si un fichero contiene una cadena de texto. El programa debe recibir dos argumentos, que serán un nombre de fichero (primer argumento) y una cadena de texto a buscar (segundo argumento). El programa creará un proceso hijo, que deberá mutar al comando '`grep -q CADENA FICHERO`'. La utilidad `grep` devolverá un valor de finalización igual a cero cuando la cadena se encuentre en el fichero, y distinto de cero cuando la cadena NO se encuentre en el fichero (o el fichero no exista).

El padre deberá recoger el valor de finalización de `grep`, y en función del mismo, imprimir por pantalla si el fichero contiene la cadena. Además, el programa deberá finalizar con un valor de 0 cuando el fichero contenga la cadena, y de 1 en caso contrario.

Ejemplos de salida:

```
$ ./contains contains.c main
El fichero contains.c contiene main
$ ./contains contains.c jamon
El fichero contains.c NO contiene jamon
```

ATENCIÓN! Este ejercicio se evaluará en función de la ejecución correcta del programa: nota del 100% si funciona según sus especificaciones; del 75% si hay algún error no relacionado con la asignatura; o del 0% si hay algún error relacionado con aspectos de sistemas operativos.

Ejercicio 2 (3 puntos) – Múltiples búsquedas

Haz un nuevo programa, llamado **ncontains.c**. Este será una extensión del anterior, pero aceptará como argumentos un número variable de nombres de fichero (como mínimo uno) y una cadena a buscar como último argumento. Este programa creará tantos hijos **de forma secuencial** como nombres de fichero tenga en los argumentos, y cada hijo se encargará de procesar un fichero.

Cada uno de los hijos creados por el proceso padre también creará otro proceso, que mutará al programa **contains**, enviando como argumentos el fichero que le corresponda y la cadena a buscar, y recogerá su valor de finalización.

En caso de que un proceso nieto retorne con un 0 (se ha encontrado la cadena), el proceso hijo correspondiente le enviará un SIGUSR1 al proceso padre. En caso de que retorne 1, le enviará un SIGUSR2. En ambos casos, el proceso hijo terminará con un código de finalización igual a cero.

El padre, por su parte, después de crear cada hijo, deberá esperar a que le llegue un SIGUSR1 o SIGUSR2, **en una espera activa**. Ambos signals deberán tratarse utilizando una única rutina de tratamiento, y el padre deberá tenerlas bloqueadas durante la ejecución del programa, excepto

cuando esté esperando. Después de recibir uno de los signals, esperará a la finalización del hijo y creará el siguiente. Al finalizar todos los hijos, el padre imprimirá por pantalla cuántos de los ficheros contenían la cadena buscada (es decir, el número de SIGUSR1 que hayan llegado).

Ejemplo de salida:

```
$ ./ncontains ncontains.c contains.c main
El fichero contains.c contiene main
El fichero ncontains.c contiene main
Contienen: 2.
$ ./ncontains ncontains.c contains.c jamon
El fichero ncontains.c NO contiene jamon
El fichero contains.c NO contiene jamon
Contienen: 0.
```

Ejercicio 3 (4 puntos) – Código Morse

Para este ejercicio deberás crear dos programas diferentes: **enviar.c** y **rebre.c**.

El objetivo del programa será enviar una cadena de texto al programa rebre, usando solo signals. Para hacerlo, usaremos un protocolo parecido al código Morse, pero aprovechando la codificación binaria, con una capacidad de enviar una letra cada ocho segundos.

Los dos programas deberán esperar en varias ocasiones a recibir un signal. **Todas las esperas deben ser bloqueantes.**

Qué debe hacer cada programa:

El programa **enviar.c** recibirá como argumentos el PID del programa rebre.c y una cadena. El programa entonces hará lo siguiente:

- Por cada letra de la cadena, el programa tomará el carácter, y enviará bit a bit su contenido al receptor. Podrá enviar:
  - SIGUSR1 para señalar un 0
  - SIGUSR2 para señalar un 1
  - SIGALRM para señalar el **final de una letra**
- Recordad que cada letra tiene ocho bits. Hay que enviar los **bits de más peso primero**. Para saber si el bit de más peso de un carácter c es 1 podéis usar la siguiente condición: `if (c & 0x80)`
- Para tratar el siguiente bit, se pueden desplazar los bits de un carácter hacia la izquierda, usando: `c = c << 1`
- El máximo que se podrá enviar, para evitar perder signals, es **un bit por segundo**. Es decir, vuestro programa deberá esperar un segundo después de cada envío, programando una alarma.
- Al acabar de enviar toda la cadena, el programa enviará un SIGINT al receptor, causando la finalización de este.
- **Recuerda**: deberás esperar siempre un segundo entre cada dos signals que envíes, para evitar que lleguen fuera de orden. Esto incluye entre cada SIGUSR1/2, pero también entre estos y el SIGALRM.

El programa **rebre.c** no recibirá ningún argumento. Deberá esperar a recibir un SIGUSR1, SIGUSR2, SIGALRM o SIGINT. Este programa irá recibiendo los signals de enviar.c, y construyendo los caracteres con los signals recibidos. Hay que gestionar de manera adecuada la máscara de signals para evitar que se pierdan.

El programa inicializará un carácter a 0, y esperará a recibir alguna de los signals:

- Cuando reciba un SIGUSR1, desplazará el carácter a la izquierda una vez.
- Cuando reciba un SIGUSR2, desplazará el carácter a la izquierda, y pondrá el bit de menos peso a 1 (`c = c | 1`).
- Cuando reciba un SIGALRM, imprimirá el carácter por pantalla, y lo volverá a inicializar a 0.
- Cuando reciba un SIGINT, finalizará. No hace falta reprogramar el SIGINT, ya que este es su tratamiento por defecto.

Ejemplo de salida:

```
[En una terminal]
$ ./enviar 36802 hola

[En otra terminal]
$ ./rebre
h
o
l
a
```

Qué se debe hacer

- El Makefile
- Los códigos de los programas en C
- La función Usage() para cada programa que sea necesario

Qué se valora

- Que se sigan las especificaciones del enunciado
- Que el uso de las llamadas al sistema sea correcto
- Que se comprueben los errores de todas las llamadas al sistema
- Que el código sea claro y correctamente indentado
- Que el Makefile tenga bien definidas las dependencias y objetivos
- Que la función Usage() muestre por pantalla cómo se debe invocar correctamente el programa en caso de que los argumentos recibidos no sean los adecuados

Qué hay que entregar

Un único fichero tar.gz con el código de todos los programas y el Makefile:

```
tar czvf clab.tar.gz Makefile *.c
```